- Logarithm functions, so log returns the logarithm of the value and has an inverse method exp, which returns e raised to that value. The sibling log methods log10 and log2 are also available and return the the log of the argument in the base of the method name.
- A related method, frexp(x), which returns a two-element array with the fraction and exponent base two of that number, so the return value is [y, z], where x = y * 2**z. The inverse method ldexp(y, z), returns the x value in that equation.
- A method hypot(x, y), which takes two arguments and returns sqrt(x**2, y**2)—in other words, the hypotenuse of a right triangle with sides x and y.
- Error and gamma functions erf, erfc, gamma, and gammac. We're just going to assume that if you know what these are, you'll know what they do.
- Lots of trig functions, including the basic sin, cos, and tan. The input value is in radians and for sin and cos must be between -1 and 1. There's also the inverse trig functions acos, asin, atan, and atan2. Finally, the hyperbolic and inverse hyperbolic functions cosh, sinh, tanh, acosh, asinh, and atanh are all also there, and all also take arguments in radians. Some of these methods have their own range limits.

Numbers

We talked about numeric literals in Integer and Floating-Point Numbers, on page 434. Here, let's talk about the library methods for numbers.

Numeric Class

All number classes inherit from the class Numeric. If you want to define your own number subclass, it is recommended that you also inherit from Numeric because Numeric does some internal things about storing numbers in memory that are useful to have.

It might go without saying—but it's our job to say it—that Numeric implements <=> and Comparable. As with other <=>, the return value is -1 if the left side is smaller, 1 if the left side is greater, and 0 if the two are equal. The equality test is also aliased as eql?.

It's also our job to say that the actual arithmetic operators—+, -, *, /, and **—are not defined by Numeric, but are defined by the subclasses individually, presumably for performance reasons. The Numeric class does define unary - for negating a number, and unary +, which is basically a no-op. Numeric also defines %, which does modular arithmetic, though this is also overridden in most of the subclasses. And Numeric defines abs for the absolute value of the number.

The basic mechanism of converting between number types for the purposes of doing arithmetic is coerce. The coerce method takes another numeric argument and returns a two element array with the argument as the first element and the original receiver as the second. The two numbers are in a common type—broadly, an integer and a non-integer argument will both be converted to floats. A float receiver and pretty much any argument will be converted to floats. A rational or complex receiver will convert an integer argument, but will be converted to float by a float argument, see Numeric Coercion, on page 368 for more examples.

Numeric defines ceil, floor, round, and truncate for the common mathematical definitions. All of these take an optional argument that is the number of digits beyond the decimal point to keep; the default is 0. So, ceil returns the smallest number above the receiver at that precision, floor returns the largest number below, round returns whichever of floor or ceil is closer, halfway point rounds up, and truncate cuts off the number.